

IDEA LAB Study

5장. 합성곱 신경망(CNN)

발표자: 안동균
(지도교수: 양선웅)

한양대학교 ERICA 로봇공학과



Intelligent Design
for Engineering
Applications Lab

[Conv → ReLU → Pool] 블록을 반복해 특징 추출, Flatten → 완전연결층 → Softmax 후 최종결과

- 이미지전체를 한번에 계산하는것이 아닌 국소부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망

층별 역할 분담

특징 추출 (① 입력층 ~ ③ 풀링층)

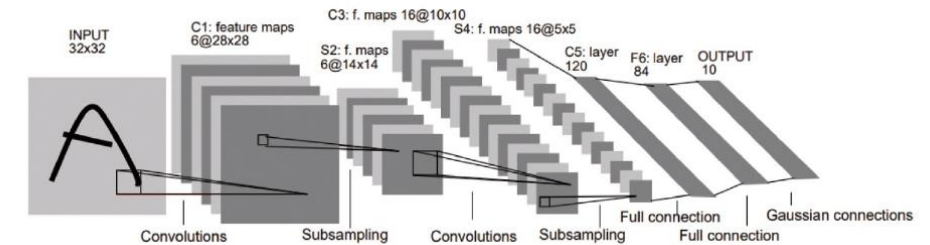
- ① 입력층: 원본 이미지가 들어오는 곳 (예: 28×28 이미지)
- ② 합성곱층(Conv): 커널로 특징 추출 → 커널 수 = 출력 채널수(feature 수) (+ReLU로 비선형성)
- ③ 풀링층(Pool): 특징 벡터 요약·축소 및 추출

분류 (④ 완전연결층 ~ ⑤ 출력층)

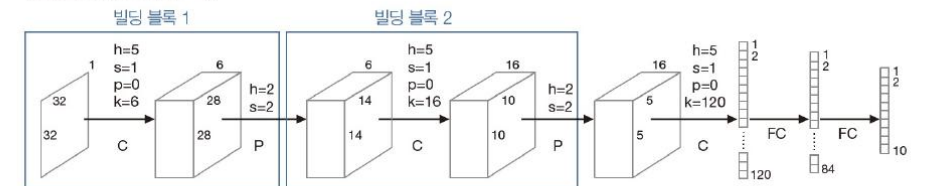
- ④ 완전연결층(Fully connected): 특징 맵을 3차원 벡터에서 1차원벡터로 펴서(Flatten) 특징 조합
- ⑤ 출력층: 클래스별 점수 산출 → 최종 분류
→ '이미지에서 특징추출시에는 3차원으로 위치정보, 특징 추출 후에는 Linear가 분류에 적합'

학습 방법

forward → loss(CrossEntropy) → backward → update(Adam)
새로운 것은 층의 종류뿐 — 학습 원리는 ANN코드와 동일



(a) [LeCun1998]의 그림



(b) 이 책의 방식에 따른 그림

그림 6-11 LeNet-5의 구조(h는 커널의 크기, s는 보폭, p는 덧대기, k는 커널 개수)

그림 6-11 LeNet-5의 구조 — 그림 출처: <파이썬으로 만드는 인공지능>(한빛아카데미) 강의자료

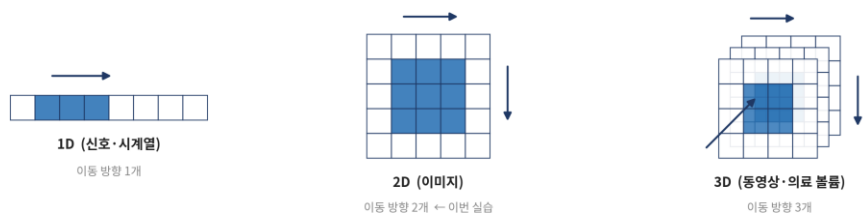
커널 1장이 특징 맵 1장을 만듦 (커널 수 = 채널 수). 층이 깊어질수록 선·모서리 → 무늬·질감 → 눈·귀 같은 형태 순으로 점점 복잡한 특징 추출

커널(필터)이 이미지를 훑으며 feature 추출

- 연산 결과 = 특징 맵(feature map): 그 특징이 이미지 어디에 얼마나 있는지의 지도



그림 6-4 컨볼루션 연산 — 그림 출처: <파이썬으로 만드는 인공지능>(한빛아카데미) 강의자료



합성곱 연산

입력이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더함
예) $2 \times (-1) + 9 \times 0 + 9 \times 1 = 7$

용어 정리

커널(필터): 특징 탐지기, 보통 3x3 홀수 크기 (W,H,D) 가로 세로 깊이(채널)

스트라이드: 커널이 움직이는 간격(1이면 한칸씩 2면 두칸씩 이동)

특성 맵: 이미지 특징의 위치정보를 담고있음(커널 1장 → 지도 1장)

커널 1개 = 특징 1종류. 커널이 많아질수록 특성 맵도 여러 장(채널이 깊어짐)

예: $[-1 \ 0 \ 1]$ 커널 → 좌우 밝기 차를 재는 수직 엣지 탐지기

커널 값은 사람이 정하지 않고 학습으로 정해지는 weight

1D · 2D · 3D 합성곱 — 데이터 형태에 따라

1D: 커널이 좌우로만 움직일 수 있음(필터가 한방향으로만 움직임)

2D: 커널이 두방향으로 움직일수있음 (두 방향 이동) — 그림 오른쪽

3D: 커널이 세방향으로 움직일수 있음 (W,H,L) → (k,k,d) → (W,H,L)

원리는 전부 동일 — 커널이 움직이는 방향의 개수만 다르다

추출한 feature를 풀링층을 통해 다운 샘플링하고 완전 연결층 softmax를 통과한후 결과값

풀링층과 출력층

풀링 (Pooling)

커널 안의 값을 최댓값(최대 풀링) 또는 평균으로 요약 → 크기 축소
지나친 상세함을 줄여 주요 특징을 추출하여 효율적으로 만들 연산량을
감소시킴 - (대충 왼쪽 하단에 어떤정보가 있다)
풀링은 고정연산 커널과 달리 정해진 규칙으로만 동작

출력층 — 분류의 마무리

완전연결층이 조합한 특징 → 클래스별 점수 산출

Softmax: 점수를 '합이 100%인 확률'로 변환

예) 점수 [2.0, 5.0, 1.0] → 확률 [5%, 90%, 3%] → 가장 높은 확률 = 최종
예측

원리: 각 점수에 exp를 씌워 비율로 나눔 → 큰 점수일수록 확률이 확 커짐

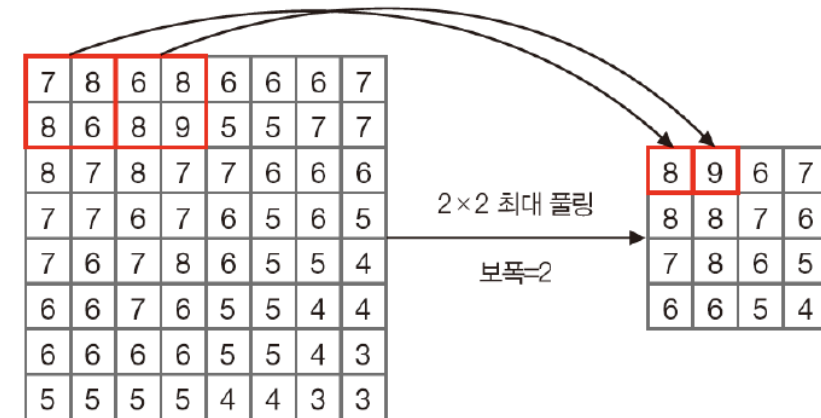


그림 6-8 최대 풀링(보폭이 2인 경우)

이미지를 완전연결층에 넣으면 위치 정보가 파괴되고 파라미터가 폭발한다

- 완전연결층으로만 구성된 심층신경망보단 CNN이 이미지 영상 처리에는 알맞음

완전연결층(nn.Linear)의 문제

공간적 구조 무시

이미지를 1차원으로 펴야(flatten) 입력 가능 → 이웃 픽셀 관계가 사라짐 (위치정보)

특징(엣지·질감)은 픽셀 하나가 아니라 픽셀들의 관계에 있음

파라미터 폭발

28×28 이미지 → Linear(784, 64)만 해도 50,240개
모든 픽셀을 전부 연결 → 큰 이미지에선 수백만 개 이상
(파라미터가 많을수록 과적합 위험도 증가)

CNN의 해법

1. 공간 정보 보존

이미지를 1차원으로 붕괴시키는 FC층과 달리, 2D 구조를 그대로 유지
픽셀 간의 인접성과 시각적 맥락(윤곽선, 형태)을 파괴 없이 학습

2. 가중치 공유로 연산 최적화

하나의 필터를 이미지 전체에 재사용하여 파라미터(가중치) 수 급감
과적합(Overfitting)을 방지하고 실시간 처리가 가능한 가벼운 모델 구현

3. 이동 불변성 (Translation Invariance)

필터가 전체 화면을 훑으며 특징을 추출
어떤 한 특징이 이미지 어디에 위치해있던 동일하게 훑고 지나가 특징을 찾을 수 있다

4. 인간의 시각을 닮은 계층적 특징 추출

얕은 층: 선, 모서리 등 단순한 특징 추출
깊은 층: 복잡한 부위 특징 추출

정리: 그래서 이미지 처리에는 CNN을 사용한다

심층 신경망(DNN): 이미지를 펴서 분류하다

● 코드 흐름

Part 1 Introduction

```
out = input.view(-1, 784)
```

28×28 이미지를 784개로 펼침 → 공간 정보 사라짐

Part 2 Method

```
nn.Linear(784, 256) → ReLU
```

256차원으로 압축 + 비선형성 부여

Part 3 Results & Discussion

```
nn.Dropout(0.25)
```

노드 일부를 랜덤하게 꺼 과적합 방지

```
nn.Linear(256, 128) → ReLU
```

128차원으로 다시 압축 + 비선형성

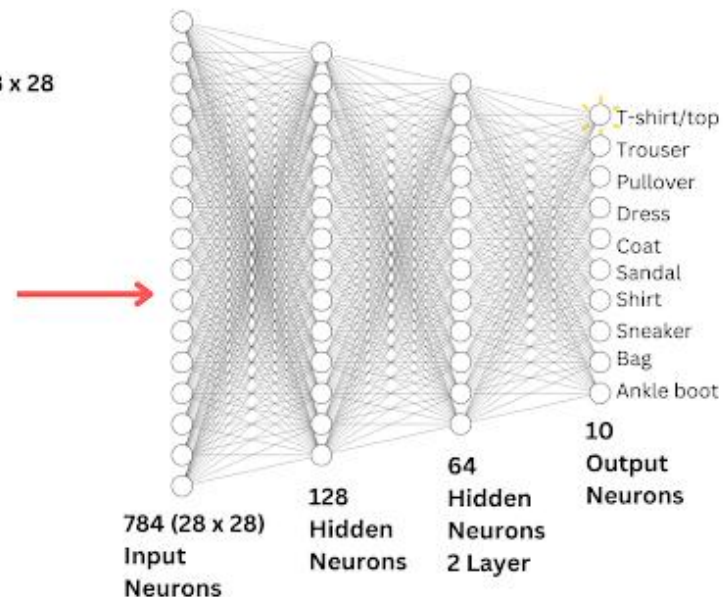
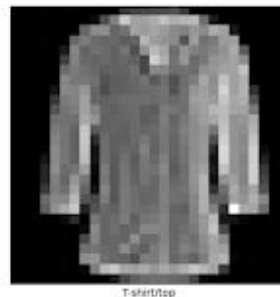
Part 4 Conclusion

```
nn.Linear(128, 10)
```

10개 클래스 점수 출력 (마지막엔 ReLU 없음)

● 네트워크 구조

MNIST Fashion dataset(28 x 28 pixels images)



입력 이미지를 펼쳐 완전연결층을 지나며 10개 클래스로 분류한다.

핵심 fully connected layer만으로 이루어진 신경망, 마지막 fc 층엔 ReLU가 없다 — 점수를 왜곡하지 않고 손실함수에 넘기기 위해.

합성곱 신경망(CNN): 공간 구조를 살려 특징 추출

Part 1 Introduction

● 코드 흐름

`self.layer1 (Conv+BN+ReLU+Pool)`
32개 필터로 특징 추출 → 32채널 14×14

`self.layer2 (Conv+BN+ReLU+Pool)`
padding 없음 → 64채널 6×6

`out.view(out.size(0), -1)`
64×6×6 = 2304로 평탄화

`fc1 → fc2 (2304→600→120)`
완전연결로 압축 + Dropout

`fc3 (120→10)`
10개 클래스 점수 (마지막엔 ReLU 없음)

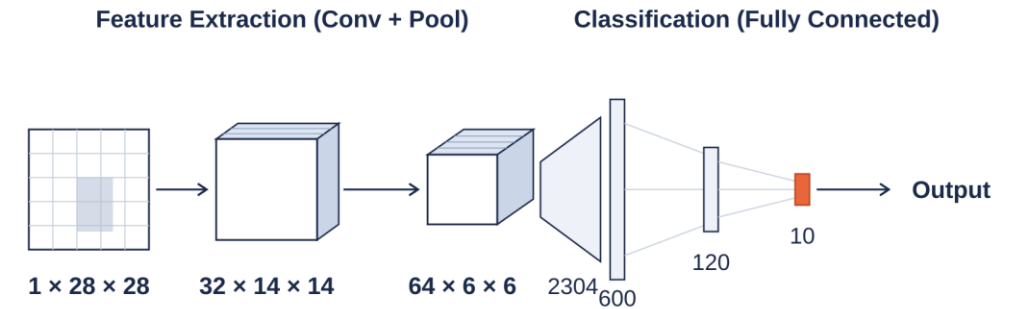
`nn.BatchNorm2d`
데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규분포화

Part 2 Method

Part 3 Results & Discussion

Part 4 Conclusion

● 특징 맵 흐름



합성곱·풀링으로 공간 구조를 보존하며 특징을 추출한 뒤 완전연결층으로 분류.

핵심 DNN과 다른 건 앞쪽 특징 추출부(합성곱)뿐 — 뒤쪽 분류부는 동일하다.

학습 루프와 손실 함수 (CrossEntropyLoss)

DNN과 CNN 모두 공유하는 부분

● 학습 루프

`outputs = model(train)`
순전파: 입력으로 예측 점수 계산

`loss = criterion(outputs, labels)`
CrossEntropyLoss로 손실 계산

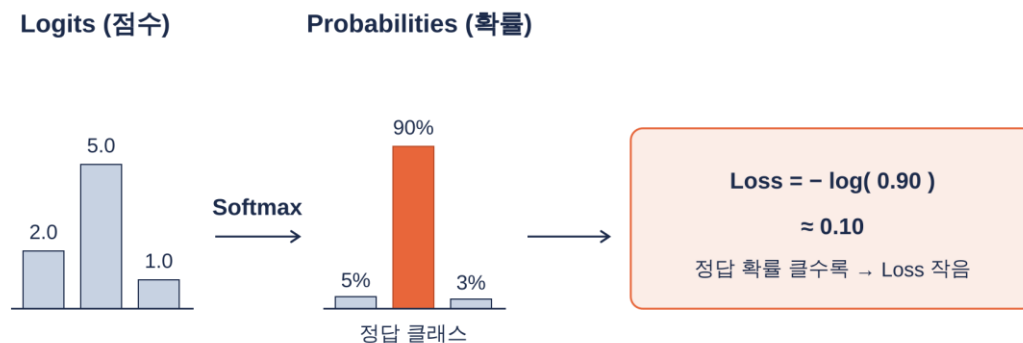
`optimizer.zero_grad()`
이전 배치의 기울기 초기화

`loss.backward()`
역전파: 각 가중치의 기울기 계산

`optimizer.step()`
기울기 방향으로 가중치 갱신

초기화→역전파→갱신을 매 배치 반복하며 학습

● 손실 계산 원리



점수를 Softmax로 확률로 바꾼 뒤, 정답 확률에 $-\log$ 를 씌워 손실을 계산.
90% → loss 0.1 (손실 가장 작음)

핵심 CrossEntropyLoss가 Softmax를 내부 처리 → 마지막 층에 ReLU를 안 쓴다.

Part 1
Introduction

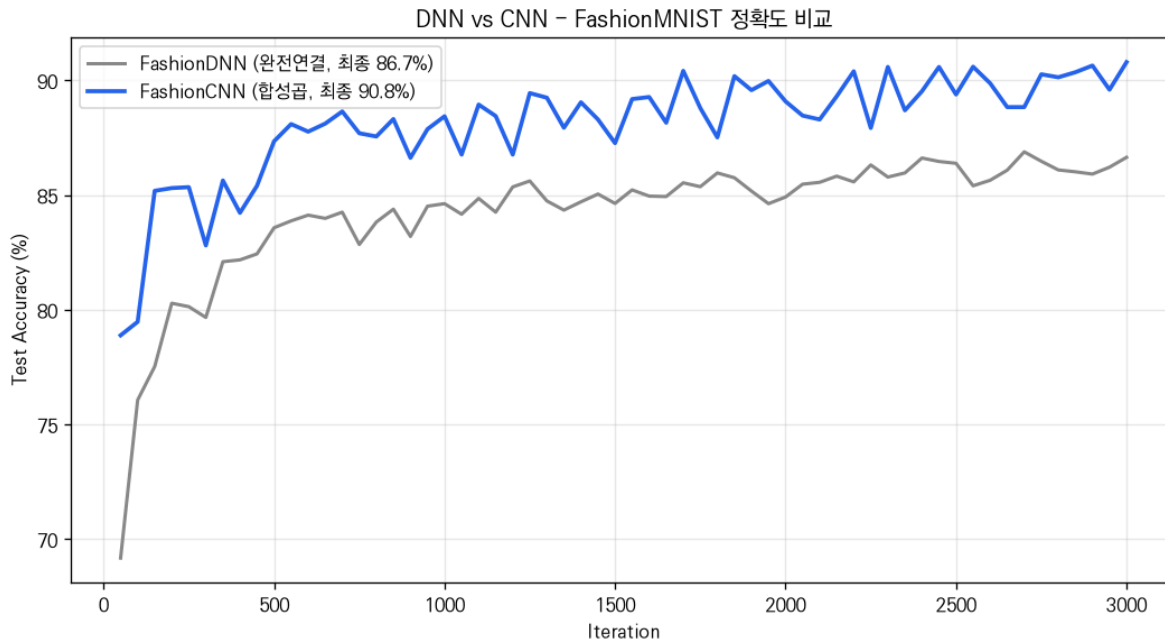
Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

결과: DNN vs CNN 정확도 비교

● 코드 실행 후 결과 비교



FashionMNIST 테스트 정확도 (반복에 따른 변화)

● 핵심 해석

86.7%

DNN (완전연결)

90.8%

CNN (합성곱)

관찰 정확도 차이는 크지 않다(86.7%→90.8%). 이것만 보면 단순한 DNN도 괜찮아 보인다.

하지만 이미지 데이터가 많아지면, 데이터를 일렬로 펴서 처리하는 DNN은 정확한 특징 추출·분류가 불가능하다.

→ 이미지 데이터에는 CNN을 써야 한다.

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

훈련된 모델의 가중치를 가져와 나의 작업에 재사용하는 모델

● 핵심 개념

사전학습 모델 활용

ImageNet 140만 장으로 이미 학습된 ResNet18의 파라미터 가져와 사용

특징 추출부 재사용

선·질감 등 모든 이미지 공통 특징은 다시 배우지 않는다

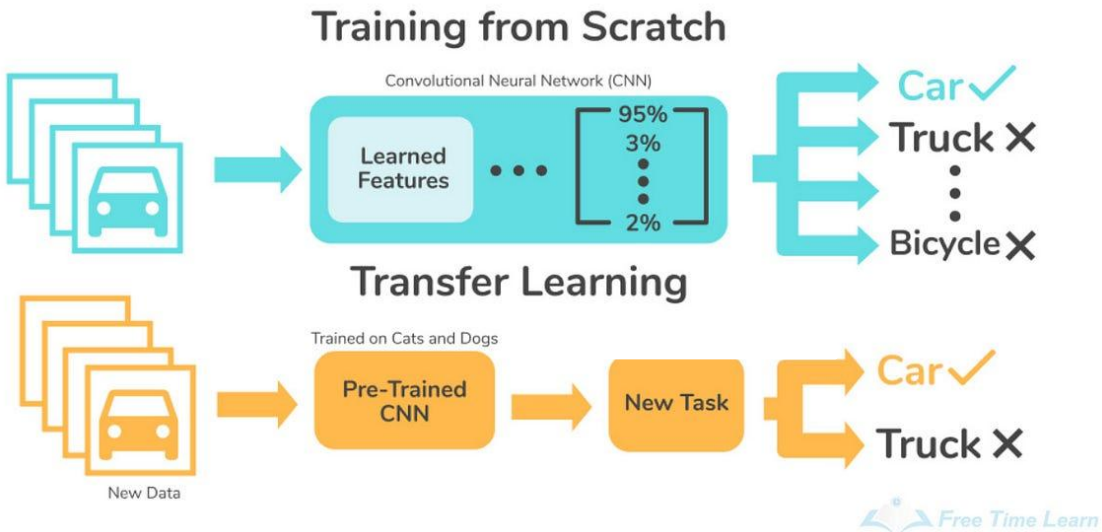
마지막 층만 교체

1000종류 출력층을 개/고양이 2종류용으로 바꿔 그 부분만 학습

적은 데이터에서 유용하게 쓸수있음

처음부터 학습보다 훨씬 적은 데이터·시간으로 높은 정확도

● 개념도



특징 추출부터 학습하는 대신, 사전학습된 모델을 새 작업에 재활용한다.

핵심 특징 추출부는 재사용, 마지막 판단층만 우리 문제(개/고양이)에 맞게 새로 학습.

5.3 코드 (1) — 데이터 준비 & 전이학습 모델

● 코드 흐름

`transforms.Compose([...])`

Resize→RandomResizedCrop→Flip→ToTensor (증강+텐서변환)→이유?

`ImageFolder(TRAIN_PATH)`

폴더명(Cat/Dog)을 그대로 정답 라벨로 인식

`resnet18(weights=DEFAULT)`

사전학습된 ResNet18 모델 로드

`requires_grad = False`

특징 추출부 전체를 얼림(freeze)

`model.fc = nn.Linear(512, 2)`

마지막 층만 개/고양이 2클래스로 교체·학습

● 전이학습 구조



사전학습된 특징 추출부는 얼리고, 마지막 분류층만 새로 학습한다.

핵심 앞부분은 모든 이미지 공통 특징이라 재사용 — 학습되는 건 fc(512→2)뿐.

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

5.3 코드 (2) — 학습 & 평가

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

● 코드 흐름

`train_model(...)` 13 epochs
전체 데이터를 13번 반복 학습

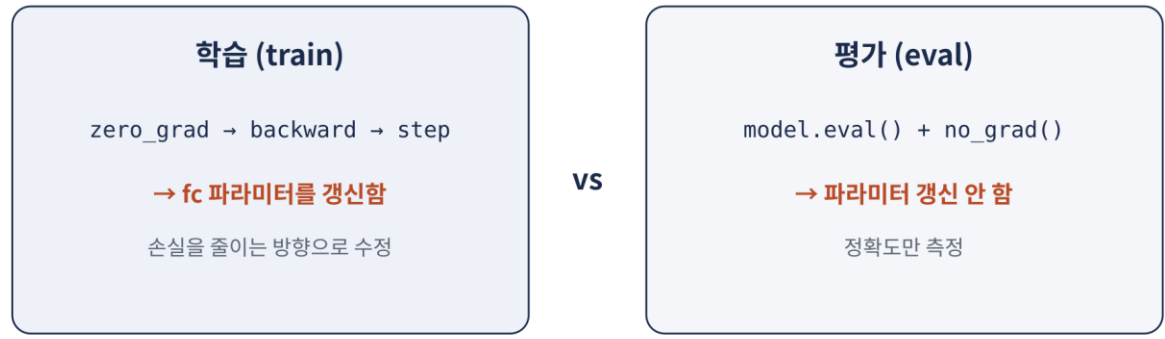
`zero_grad`→`backward`→`step`
`fc` 층 파라미터만 갱신-`weight`, `bias` (앞부분은 얼림)

`torch.save(...'.pth')`
에폭마다 모델 저장 (00~12.pth)

`eval: model.eval()+no_grad`
학습 안 한 테스트 데이터로 정확도 측정 →모델의 일반화 성능 검증

저장된 13개 모델 각각 평가
에폭이 지날수록 정확도가 오르는지 확인

● 학습 vs 평가



에폭마다 저장한 13개 모델(00~12.pth)을 하나씩 불러와 평가

정확도를 측정하는 평가 시 모델의 일반화 성능을 검증하기 위해 학습에 사용되지 않은 테스트 데이터 사용

학습은 파라미터를 바꾸고, 평가는 바꾸지 않고 정확도만 측정한다.

핵심 평가엔 `backward`·`step`이 없다 — 처음 보는 데이터로 진짜 실력을 쟀다.

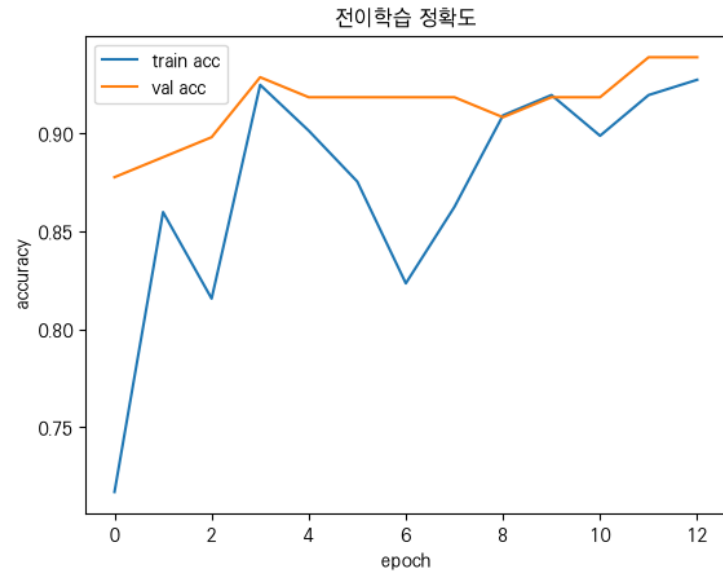
결과: 전이학습으로 개/고양이 분류

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion



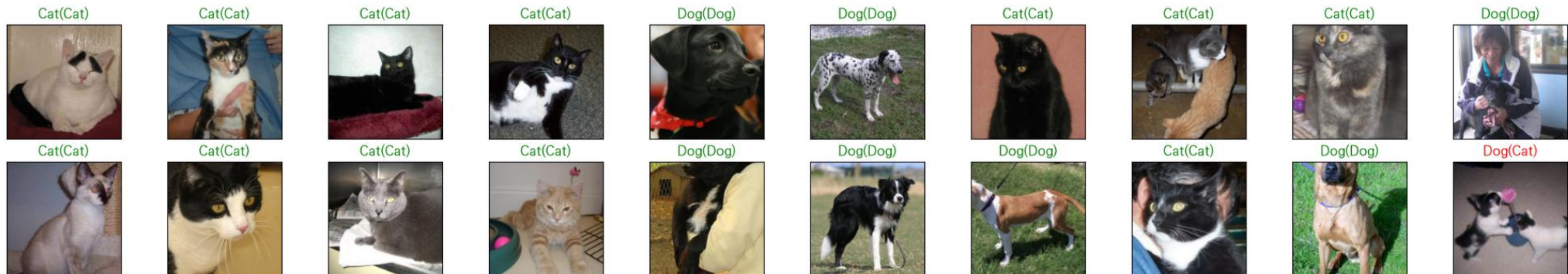
결과 해석

≈ 94%
검증(val) 최고 정확도

적은 데이터·13 에폭 만으로 높은 정확도 → 전이학습
이 잘 작동함.

학습 진행에 따라 정확도 상승 (검증 ≈ 94%)

예측 결과 · 초록 = 정답, 빨강 = 오답



특성 추출 vs 파인튜닝 (Fine-Tuning)

● 특성추출과 파인튜닝 무엇이 다른가?

Part 1
Introduction

특성 추출 (Feature Extraction) 5.3 코드

방식 앞부분 전부 얼리고 완전연결층 가중치만 학습

장점 빠름, 적은 데이터 OK, 과적합 낮음

단점 성능 천장이 낮음

Part 2
Method

파인튜닝 (Fine-Tuning) 앞부분까지 함께 재학습

방식 앞부분(특징추출부분)도 함께(작은 lr로) 재학습

장점 성능 천장 높음·특수 이미지 대응

단점 데이터 多 필요·느림 → 파라미터 많아짐 → 망가질 위험

Part 3
Results &
Discussion

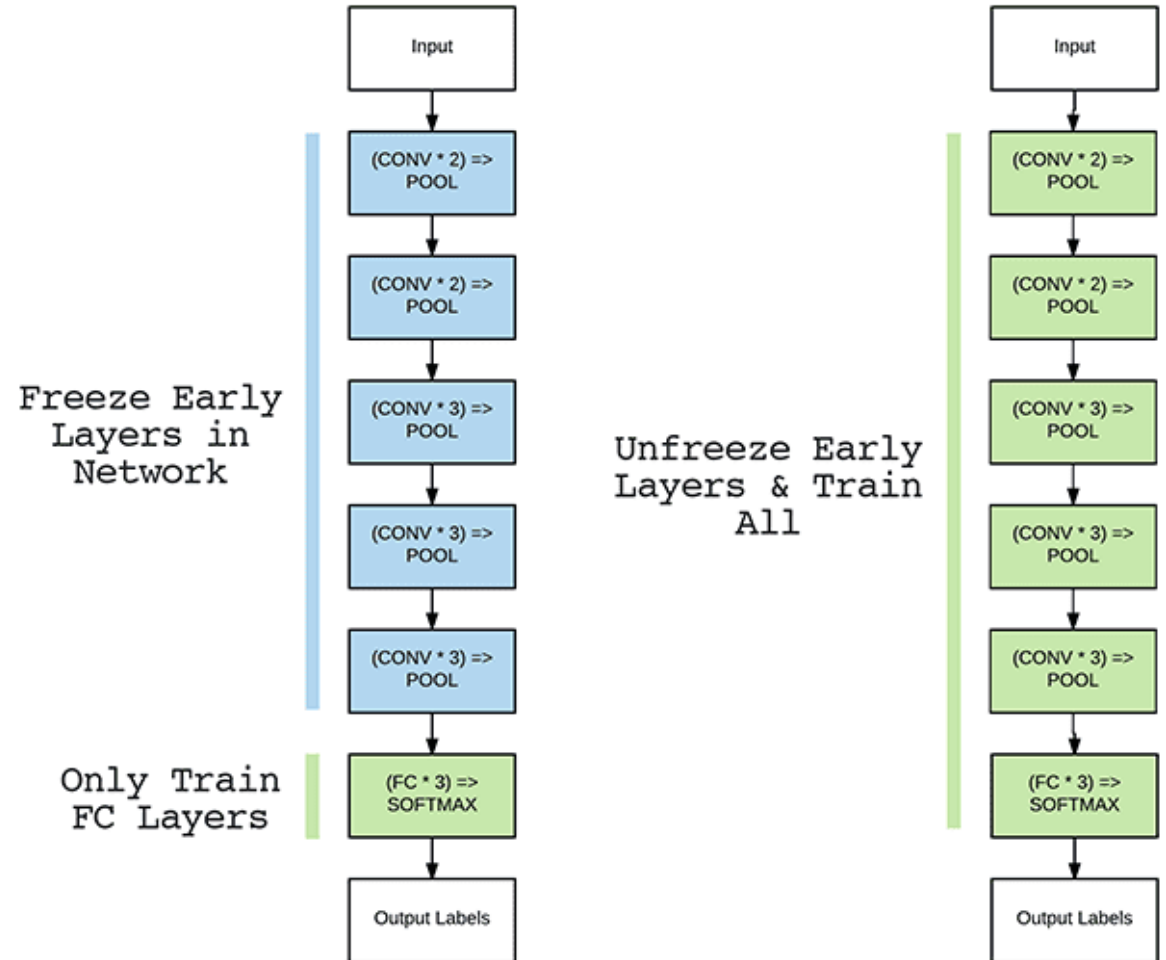
Part 4
Conclusion

데이터 적음·유사 → 특성 추출(데이터 분류기만 학습)

데이터 적음·특수 → 파인 튜닝(합성곱 일부분과 데이터 분류기 학습)

데이터 많음·유사 → 파인 튜닝

데이터 많음·특수 → 특성 추출



왼쪽 = 특성 추출 · 오른쪽 = 파인튜닝

특성 맵 시각화: CNN 블랙박스를 열어본다 (XAI)

Part 1 Introduction

Part 2 Method

Part 3 Results & Discussion

Part 4 Conclusion

● 핵심 개념

CNN은 블랙박스

내부에서 무슨 특징을 보는지 사람이 알기 어렵다, 어떻게 판단했는지 근거를 알기 어려움

설명 가능한 AI (XAI)

결과의 신뢰성을 위해 내부 동작을 들여다봐야 한다

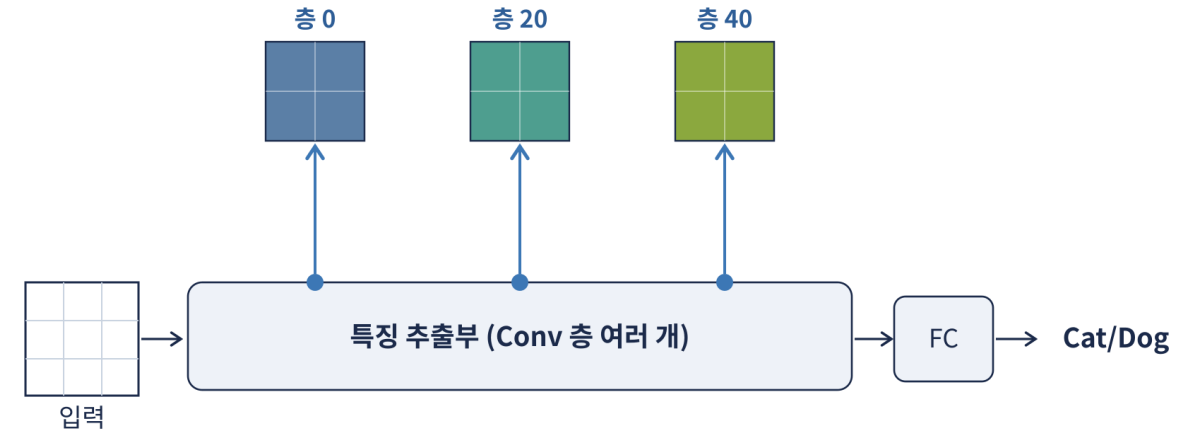
특성 맵 시각화

각 층의 출력(특성 맵)을 꺼내 이미지로 확인한다

forward hook

순전파 중 지나가서 사라지는 중간 층의 입력 및 출력값을 가져오는 것-시각화를 위해서 필수적

● 방법: hook으로 중간 층 엿보기



forward hook — 순전파 중 중간 층의 출력(특성 맵)을 가로챈다

층 0·20·40 세 지점에서 특성 맵을 꺼내 어떻게 변하는지 관찰한다.

핵심 새 모델을 만드는 게 아니라, 이미 있는 CNN의 '속'을 들여다보는 작업이다.

5.4 코드 — forward hook으로 특성 맵 가져오기

● 코드 흐름

class XAI
features(Conv 여러 층) + classifier(FC) 로 구성

Log_softmax()
소프트맥스 함수에 log값을 취한 연산 → 수치적 안정성(일반 소프트맥스 함수는 점수에 지수를 씌우는거라 값이 매우 커질수있음)
(+책에는 기울기 완화)

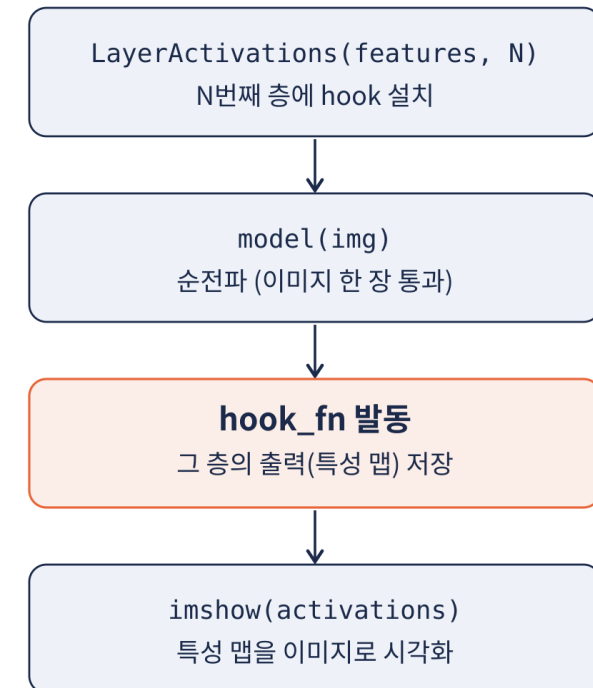
register_forward_hook(fn)
순전파중 지정한 층의 출력을 가져오는 것

LayerActivations(features, N)
N번째 층(0/20/40)에 hook 설치

img = ToTensor()(img).unsqueeze(0)
ToTensor() 이미지값을 텐서로 변환 (H,W,C)→(C,H,W)
.unsqueeze(0) 모델이 4차원을 → 맨 앞에 배치 차원 추가

imshow(activations[0][k])
저장된 특성 맵을 이미지로 그림

● hook 데이터 흐름



Part 1
Introduction

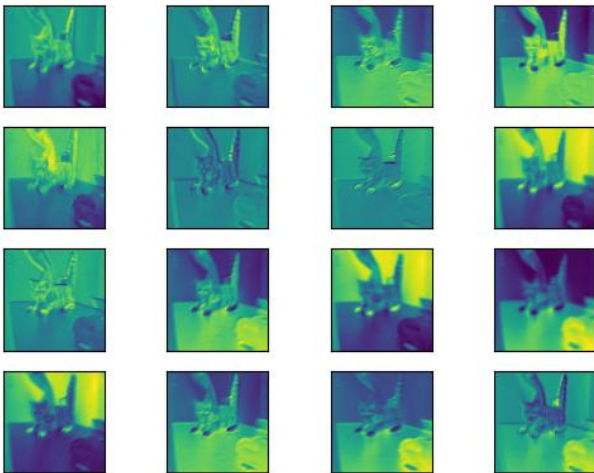
Part 2
Method

Part 3
Results &
Discussion

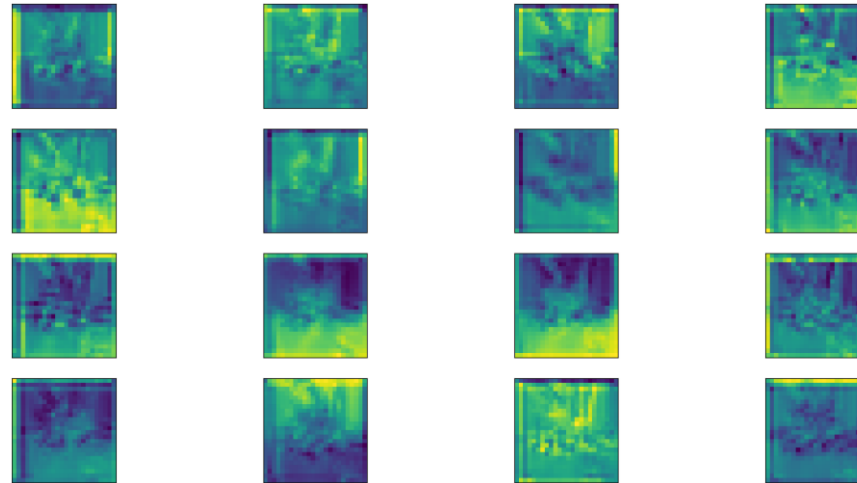
Part 4
Conclusion

층별 특성 맵 — 형태에서 추상으로

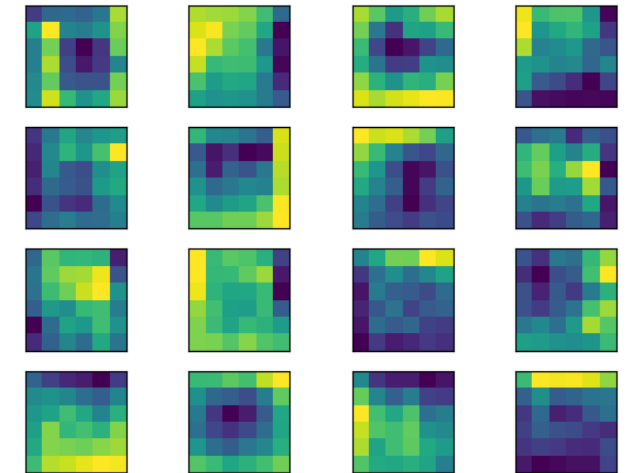
- 층이 깊어질수록 원본 형태는 사라지고 특징만 남는다



층 0 (얕음)
고양이 형태가 그대로 유지



층 20 (중간)
형태가 흐려지고 덩어리로



층 40 (깊음)
형태는 사라지고 추상 특징만

결론 입력층에 가까울수록 형태가 유지되고, 출력층에 가까울수록 추상적 특징만 남는다.
→ 이렇게 필터·특성 맵을 시각화해서 CNN 결과의 신뢰성을 확보할 수 있다.

Part 1
Introduction

Part 2
Method

Part 3
Results &
Discussion

Part 4
Conclusion

Any Questions?

이름 안동균

소속 한양대학교 로봇공학과

이메일 ehdrbs123450@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

